

面试押题 · Mooncake（阿里 · KVCache 组）

日期：2026-08-27

对象：阿里做 Mooncake 的组

材料来源：本地仓 `/volume/pt-train/users/zhaoye/Mooncake` (origin=kvcache-ai, fork=XFDG, HEAD `160b963`)、FAST25 论文、`doc/mooncake/` 下我自己提的 PR 记录
用法：先过 §1 架构速成建立整体图，再按 §3 深挖四题逐条背答法，§4 其余 PR 速览兜底。
面试官会顺着我改过的代码往下钻，所以每个 PR 都要能讲：现象 → 根因 → 为什么这样修 → 替代方案 → 怎么测。

0. 一句话定位自己（开场用）

我给 Mooncake 上游提过 8+ 个 bugfix PR，覆盖它的两大子系统：Store（分布式 KVCache 存储 + SSD offload）和 Transfer Engine / TENT（RDMA 传输）。改动集中在存储后端的正确性（bucket 文件覆盖、offload 空读、幂等 re-offload）、RDMA 资源生命周期（端口恢复清 stale endpoint、unregister 还原 fork 状态防 VMA 耗尽），以及传输层并发（MPSC 队列满时活锁）。都是在真实多卡 / 高 churn 场景下暴露、我做了最小复现 + 回归测试的问题。

红线：凭据（SwanLab key、OSS AK/SK、私有 token）只走平台环境变量，绝不写进任何文档 / 脚本 / git。本文件无任何凭据。

1. Mooncake 架构速成

1.1 它是什么

Mooncake 是 Moonshot AI（Kimi）的推理服务平台，开源核心是一套以 KVCache 为中心的分离式（disaggregated）架构。一句话：把 LLM 推理拆成 prefill 和 decode 两类池子，中间用一个分布式

KVCache 池 + 高性能传输层把它们连起来，让缓存能跨实例复用。论文里在真实负载下让 Kimi 多扛 75% 的请求还满足 SLO。

1.2 为什么要分离（PD 分离的动机）

prefill 和 decode 是两种截然不同的负载：

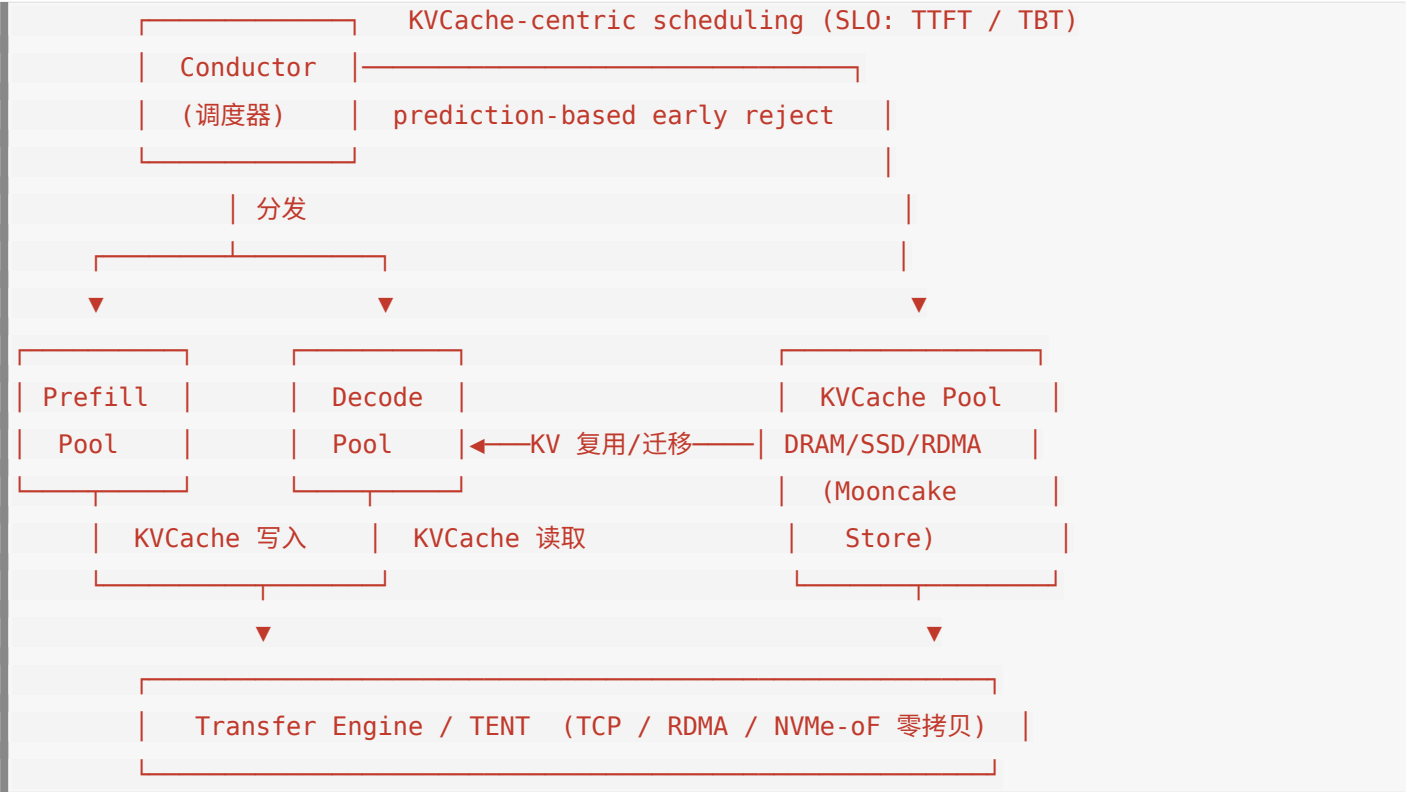
阶段	特征	瓶颈
----	----	----

Prefill 一次处理整段 prompt，算力密集，大矩阵乘 **compute-bound**

Decode 逐 token 自回归，每步只算一个 token，反复读 KV **memory-bandwidth-bound**

如果放在同一张卡上混跑，两者互相干扰：decode 的低延迟 SLO（TBT）被 prefill 的长任务打断，prefill 的吞吐又被 decode 的碎步拖累。**分离**之后各自组池、各自优化、各自扩缩容。代价是：prefill 算出来的 KVCache 必须**高效地搬到** decode 节点——这就是 Transfer Engine 存在的理由。

1.3 四大件



1. **Conductor**（调度器）：KVCache-centric 调度，平衡吞吐同时满足 **TTFT**（首 token 延迟）和

TBT (token 间延迟) SLO; 过载时用 **prediction-based early rejection** (提前预测会违反 SLO 就早拒, 而不是收了再超时)。

2. **Prefill Pool / Decode Pool**: 分离的两类执行池。

3. **Mooncake Store (分布式 KVCache 池)**: 把集群里闲置的 CPU / DRAM / SSD / RDMA 带宽汇聚成一个分布式缓存存储。热的 KV 在 DRAM, 冷的 **offload 到 SSD**。——我的大多数 Store PR 就在这一层的 SSD offload / bucket 存储路径上。

4. **Transfer Engine (传输引擎) + TENT**: 高性能数据传输库, 支持 TCP / RDMA / NVMe-oF, 零拷贝跨节点搬 KV。TENT 是它内部一套 RDMA 传输框架 (context / endpoint_store / workers / QP 管理)。——我的 TE PR 在这一层的 RDMA 资源生命周期和并发。

1.4 为什么 KVCache 能复用 (这套架构的价值内核)

多个请求常有**公共前缀** (system prompt、few-shot、同一文档的多轮问答)。前缀的 KVCache 算一次就能被后续请求共享, 省掉重复 prefill。要跨实例共享, 就得有一个**所有 prefill/decode 节点都能高速访问的分布式 KV 池**——这就是 Store + Transfer Engine 的组合价值。近期生态: vLLM 官方接了 Mooncake Store, SGLang 用 Transfer Engine 做 P2P 权重传输 (Kimi-K2 1T 模型权重更新 53s→7.2s)。

1.5 面试可能的架构追问 (速答)

- **Q: 为什么不直接把 KV 放本地显存就好?** A: 显存贵且小, 跨实例不能共享; 分离后 decode 池要读 prefill 池算的 KV, 必须有跨节点的池 + 传输。
 - **Q: KV 池放 SSD 不会慢吗?** A: 分层——热 KV 在 DRAM, 冷的才 offload SSD; SSD 命中省掉的是整段 prefill 重算, 仍然划算。offload 路径的正确性/性能正是我几个 PR 的战场。
 - **Q: early rejection 为什么要 prediction-based?** A: 收了请求再超时既浪费算力又违反 SLO; 提前预测这个请求会不会拖垮 TTFT/TBT, 早拒能保住已接请求的 SLO。
-

2. 我的 PR 全景（8 个，按子系统分）

PR	子系统	一句话	状态
#3652	Store	多 client 共享 storage_path → BucketId 种子撞车 → bucket 文件互相覆盖截断	OPEN（深挖①）
#3614	Store	offload 后 keys 列得出但读为空；真堵点是 HEAD 编译断 + 补 LRU 回读回归测试	OPEN（深挖②）
#2632	Store	Heartbeat 里同步跑 OffloadObjects 卡住心跳 → 改 async future	doc（深挖③）
#3604	TE/TENT	RDMA 端口恢复后不清 stale endpoint, QP 停在 ERR → 恢复后反而慢 5×	doc（深挖④）
#3661	TE/TENT	BoundedMPSCQueue push 满时死等 → 6 个 RDMA worker 活锁	doc（速览）
#3660	TE	unregister 不还原 MADV_DOFORK → VMA 耗尽 ENOMEM	doc（速览）
#3662	Store	batch_get_session_start 用错 replica 选择器, SSD offload 键报 702	doc（速览）
#3655	Store	BatchOffload 遇已持久化 key 整批拒 → 改幂等跳过	doc（速览）
#3659	Store	OffsetAllocator 后端 O_DIRECT 标志名不副实（fd 没设 O_DIRECT）	doc（速览）
#3602	Store	standalone client SIGTERM 绕过清理成僵尸 + 无优雅期	doc（速览）

注：本地仓 HEAD [160b963](#) 是主干附近，#3604 的 `evictAll/resume` 改动在 XFDG fork 的 `feature` 分支上，主干 `grep` 不到——面试讲到时说清楚「这是我提的 PR，改动在我 fork 分支」，别说成主干现状。

3. 深挖四题（重点背）

深挖 ① · #3652 — Bucket 文件撞车（Store 存储本质，最对口）

现象：多个 client 共享同一个 `storage_path` 时，key 通过 `get_all_keys/IsExist` 都列得出来，但读出来是空。Reporter（8 个 TP rank）观察到 8 个 rank 生成完全相同的 bucket id（`7320020267008` × 8），报错 `Read size mismatch ... got: 0`，且在任何 eviction 发生之前就出现。

根因（我核过代码，`storage_backend.cpp:1689`）：

```
BucketIdGenerator::BucketIdGenerator(int64_t start) {
    if (start <= 0) {
        auto cur_time_stamp = time_gen();
        current_id_ = (cur_time_stamp << TIMESTAMP_SHIFT) | SEQUENCE_ID_SHIFT;
    } else { current_id_ = start; }
}
```

- `time_gen()` 是整秒粒度，`SEQUENCE_ID_SHIFT = 0`，所以低 12 位恒为 0、时间戳只有 1 秒分辨率。
- 结果：同一秒内构造 generator 的所有 client 种子完全相同 → id 序列完全相同。
- bucket 文件 `<storage_path>/<bucket_id>.bucket` 用 `O_CREAT|O_TRUNC`（我核过 `storage_backend.cpp:939`）但没有 `O_EXCL` → 后构造的 client 静默 truncate 掉前一个的 bucket → key 还挂在 master 元数据里（列得出），但数据被截掉了（读为空）。

这条链的关键洞察：这是一个确定性的数据损坏，不是概率竞态——种子撞车是必然的，只要多 client 同秒起。

我的修（commit `5e58534c`）：新增 `GenerateBucketIdEntropySeed()`，把 per-process 唯一 nonce（`std::random_device` + `getpid()` + ASLR 栈地址）+ 单调计数器 折进低 12 个 sequence 位；构造函数把它 OR 进去替代原来的 `SEQUENCE_ID_SHIFT`。高位时间戳不动 → 单调性、“恢复上次用过的最大 bucket id”的顺序都不变。撞车概率降到 $\sim 1/2^{12}$ 且非确定。

为什么不选 issue 里更“完整”的方案 B（每个 bucket 的 `.meta` 里写 `owner_id`）：那是个需要 master / SGLang 协同的开放设计点（要改协议）。熵修复不改协议、自包含，无论未来选哪种 owner 模型，都先把

这个确定性损坏消掉。——这是加分点：知道 `minimal self-contained fix` vs 大改的取舍。

怎么测：核心逻辑独立 `entropy_test.cc` (g++-13) 验证——高位时间戳保留（单调）、低 12 位非零、两个 generator 发散、显式 start 路径不受影响、单调递增，全 PASS；并在 `storage_backend_test.cpp` 加 `FreshStartSeedsDivergeAcrossClients` 回归。

预判追问 & 答法：

- *「为什么不加 `O_EXCL` 就完了？」* → 加 `O_EXCL` 会把静默损坏变成显式报错，能止血但不解决"两个 client 本该用不同 id"的根本问题（加了 `O_EXCL` 后同秒起的第二个 client 直接创建失败，服务起不来）。熵修复让它们本来就不撞。两者可叠加（防御纵深），但根因是种子。
- *「random_device 够随机吗 / 会不会还撞？」* → 折进的是 random_device + pid + ASLR 三源 + 单调计数器，同秒同种子概率 $\sim 1/2^{12}$ ，且 fold 了 counter 保证同进程内连续构造也不撞。
- *「高位为什么必须保留？」* → bucket id 恢复依赖"上次用过的最大 id"来续号（`storage_backend.cpp:2116` 那套 `INIT_NEW_START_ID` 逻辑），高位时间戳是单调续号的锚，动了会破坏 restart recovery。

深挖 ② · #3614 — offload 空读，真堵点是编译断（体现"证伪能力"）

现象 (issue #3465)：eviction offload 到 SSD 后，`get_all_keys` 列得出 key，但 `get_value` 返回空。

我做的关键动作是"证伪 reporter 的假设"：reporter 把它归因为某 commit `67aaf6d` 和"六月→main 的 regression"。我查 git history 发现读路径自六月起逐字节没变（只有 test/config commit 碰过那几个文件）——这是红鲱鱼。

真正的机制：offload 的重复分桶——`GroupOffloadingKeysByBucket` 可能把上一个心跳的 carryover key 放进 >1 个桶；第一个桶提交后，后面的桶因 `OBJECT_ALREADY_EXISTS` 原子失败 → 该桶所有 key 被 NACK → master 丢掉 LOCAL_DISK replica → "列得出但空"。这个缺陷已被上游 #3479（去重 carryover key）修掉，且已在 HEAD。

但真正卡住我的不是 offload 逻辑，是 HEAD 编译断：commit `4a0debb4` (#3401) 在

`master_service.cpp:6967` 调 `metadata.PopReplicasWithCacheTotalAccounting(...)`，可 `3b5a5941` (#2409) 早把这方法从 `ObjectMetadata` 结构体里移出去变成 `MasterService` 的 static 方法了 → HEAD 根本编译不过 → `#3465` 的复现测试压根跑不起来。

我的修：① `master_service.cpp:6967` 把调用改成 static 形式

`PopReplicasWithCacheTotalAccounting(metadata, ...)` (对齐其余 11 个调用点，1 行解锁)；② 加 `BucketLruEvictionReadBackAfterEvict3465` (~130 行) 回归测试：LRU + watermark eviction 下 offload 3 个 key，读 key_0 抬它的 LRU 时间戳让 key_1 成为 victim，驱动 eviction，断言 `victim==key_1`、幸存者逐字节读回、被逐 key 的 `IsExist==false` 且 `BatchLoad` 失败 (`INVALID_KEY`) ——永不静默返回空。把 `#3465` 背后的存储后端不变式钉进测试。

怎么测：上海 CPU box, `g++-11 + -std=gnu++20` (C++20 `atomic_flag::test()` 需 GCC 11)，`-j1/-j4` (`-j16` OOM 杀 cc1plus)，全新 build dir。复现测试 PASS；`storage_backend_test` 98/99 (那 1 个 `AdaptorWatermark...Restart` 失败是预存在、无关，revert 我的改动仍失败)。

预判追问 & 答法：

- *「你怎么确定那个失败测试和你无关？」* → stash 我的改动重新编译，同一个测试同样失败；说明是 HEAD 自带，不是我引入。这是判断"flaky vs 真 bug"的标准操作：先读失败 job log，再用 **revert 对照**。
- *「既然 `#3479` 已经修了重复分桶，`#3465` 为什么还开着？」* → `#3479` 自己 PR body 都写了"修的是独立可复现的重复分桶，不声称完全解决 `#3465` 的空值症状"。我的贡献是：解锁 HEAD 编译 + 用回归测试把"列得出必可读、逐出必显式失败"这个不变式固定下来，让症状不会悄悄回来。

深挖 ③ · #2632 — 心跳里同步 offload (分布式 SLO 意识)

现象：`FileStorage::Heartbeat()` 的 STEP 3 同步调用 `OffloadObjects()`。大批量时，把每个 bucket 写 SSD 要 O(秒)级，卡在心跳线程上。一旦心跳超过 `client_ttl`，master 判定 client 过期、把它的 offload 任务全清掉，每个 key 打 `"Offloading task expired"` 日志——offload 工作被静默丢弃，SSD 命中率下降。

根因：心跳 (master 的存活探测 RPC) 和重活 (写 SSD) 共用同一个线程。心跳的本质是"我还活着"的低延

迟信号，绝不能被 O(秒) 的 IO 阻塞。

我的修：把 `OffloadObjects` 派发到一个 `std::future<void> offload_future_`，心跳线程排队后立即返回。

最多一个 offload 在飞：心跳 tick 若发现上一个 future 还在跑就跳过 STEP 3，让 master 下个 tick 重新排这些 key。析构时 wait 这个 future 再拆存储后端，防对后端指针 use-after-free。`storage_backend.cpp` 里 `BatchOffload/EvictToMakeRoom` 的单写者不变式保留——`offload_future_` 取代原来的 `heartbeat_thread_` 做串行化。

预判追问 & 答法：

- *「异步了会不会两个 offload 并发写坏？」* → 不会，我保证最多一个在飞（in-flight 时跳过），单写者不变式没破。这正是"async 但仍串行"的关键设计。
- *「跳过的 key 丢了吗？」* → 没丢，master 下个心跳会重新把它们排进来（offload 任务在 master 侧是可重排的）。
- *「为什么不干脆开个专门的 offload 线程池？」* → future 方案最小、够用：offload 本身要串行（单写者），不需要并发池；用 future 只是把它挪出心跳的关键路径。加线程池反而要处理更多同步。
- *「析构 use-after-free 怎么来的？」* → future 里跑的 offload 持有 storage backend 指针，如果析构不 wait 就先释放了 backend，飞行中的 offload 就访问悬垂指针。所以析构必须 join。

深挖 ④ · #3604 — RDMA 端口恢复不清 stale endpoint (Transfer Engine 最对口)

现象：RDMA 端口 down 再 up 后，传输卡住，而且反直觉地比断网期间还慢 5×（reporter 7-rail: 5.48s vs 断网时 1.03s），持续 20s 超时。

根因：`IBV_EVENT_PORT_ACTIVE`（端口恢复事件）只调了 `context->resume()`，它只翻转一个 `DeviceStatus` 标志。可是端口 down 期间进入 `IBV_QPS_ERR` 状态的 QP 从没有被销毁或重建。endpoint store 继续返回这些 **stale endpoint**，work request 被 post 进死 QP、永不 complete → 传输停滞。（为什么比断网还慢：断网时请求快速失败重试，恢复后却往死 QP 里灌、卡到 20s 超时。）

我的修（4 文件，+60/-1）：给 `EndpointStore` 加纯虚 `evictAll()`，`FIFOEndpointStore` 和

`SIEVEEndpointStore` 各实现——把 active map 里每个 entry 挪到 waiting list 并 `beginDestroy()`，让 QP 进 ERR 态、硬件把 inflight WR flush 到 CQ。`RdmaContext::resume()` 翻转状态回 `DEVICE_ENABLED` 后调 `evictEndpoints()`。下一次 `getOrInsert()` 给每个 peer 建全新 endpoint + 新 QP，恢复满吞吐。

关联 bug（体现 review 深度）：这个 PR 的 CI 一开始红，我读 job log 发现是真编译错不是 flaky——`SIEVEEndpointStore` 因为基类 `evictAll()=0` 是纯虚、而它头文件里漏了 `evictAll()` 声明（只有 .cpp 实现）→ `invalid new-expression of abstract class type`。补上 `void evictAll() override;` 声明（commit `bd2e9d3`）。

预判追问 & 答法：

- *「为什么 `evictAll` 而不是只重建坏的那几个 QP？」* → 端口 flap 后无法可靠区分哪些 QP 还活着；全清 + 惰性重建（`getOrInsert` 按需建新的）最简单且正确，代价是恢复后第一次访问每个 peer 会重建一次，可接受。
- *「`beginDestroy` 之后 inflight 的请求怎么办？」* → 转 ERR 态会让硬件把 inflight WR 以 error 形式 flush 到 CQ，上层能感知失败并重试到新 QP，而不是永远等一个死 QP。
- *「FIFO 和 SIEVE 两个 store 是什么？」* → endpoint 的缓存淘汰策略（FIFO / SIEVE 算法），`evictAll` 要在两个实现里都提供，否则用 SIEVE 的路径没法全清。
- *「CI 红你怎么判断是真 bug？」* → `gh run view --log-failed` 读日志看到抽象类实例化错误，不是超时/环境问题；主干绿 ≠ 我的 PR 没引入真 bug。

4. 其余 PR 速览（兜底，被问到能接住）

- #3661 · MPSC 队列活锁（并发功底）：`BoundedMPSCQueue::push()` 满时（`dif<0`）`std::this_thread::yield()` 死等，而 worker 又把重试的 slice 重新塞回它自己要 drain 的这个满队列 → work-conservation 违反：一个满队列 wedge 住全部 6 个 RDMA worker，静默活锁。修：加非阻塞 `try_push()`（满则 `return false`），`Workers::submit()` 用它、满时返回 `Status::kTooManyRequests`；三条 worker 重试路径检查返回值、失败就把 slice 标 `FAILED` 上抛。核过代码：`push`(64 行) 满时 `yield`，`try_push`(45 行) 满时直接 `return false`，同一个 Vyukov 环形队列的 CAS。

- 追问*「为什么不加大队列？」*：加大只是推迟活锁；根因是"往必须 drain 的队列里回灌"，得让满时可失败并上抛，不是扩容。（HEAD 上确实有一个"加大容量减少 kTooManyRequests"的后续 commit，但那是缓解，不是根治。）
- #3660 · VMA 耗尽（RDMA + OS 内存）：`ibv_fork_init()` 后 `ibv_reg_mr` 会对注册区 `madvise(MADV_DONTFORK)`，但 `unregister` 路径从不还原 `MADV_DOFORK` → VMA 分裂永不愈合，高频 `register/unregister`（多模态每请求注册 buffer）耗尽 `vm.max_map_count` → 后续 `ibv_reg_mr/ibv_create_qp` 全 `ENOMEM`。修：两条 `unregister` 路径（`rdma_transport` + `TENT`）在 `ibv_dereg_mr` 成功后补 `madvise(addr, len, MADV_DOFORK)`，还原失败只 warn（`dereg` 本身已成功）。
- 追问*「`MADV_DONTFORK` 干嘛的？」*：RDMA 注册内存不能被 fork 的子进程 COW 破坏物理页映射，所以标 `DONTFORK` 让子进程不继承这段；但 MR 销毁后该段普通内存应恢复可 fork/可合并，否则 VMA 一直碎。
- #3662 · replica 选择器用错（SSD offload 正确性）：`batch_get_session_start` 用 `SelectCompleteMemoryReplica`（只选内存 replica），SSD offload 场景下数据只在 SSD → `INVALID_REPLICA`(702)。修：改用 `SelectBestReplica`（处理 memory/SSD/disk/DFS 所有类型），对齐 `batch_get_into` 等其它 get 方法。1 行。
- 追问*「为什么别的方法没这问题？」*：`batch_get_into` 早就用对了 `SelectBestReplica`，是 `session_start` 这条路径遗漏，属一致性 bug。
- #3655 · 幂等 re-offload：`BatchOffload` 遇到已持久化的 key 会整批拒绝。修：跳过已持久化 key 而不是拒整批，给 `PendingEviction` 加 `skipped_keys` 字段，commit 阶段优雅处理重复。
- #3659 · `O_DIRECT` 名不副实：`OffsetAllocatorStorageBackend` 三处 `use_direct_io=true` 但 fd 从没设 `O_DIRECT`、向量 IO 路径也没对齐处理 → 标志误导。改成 `false`。
- #3602 · `SIGTERM` 僵尸：standalone client 的 `pthread_sigmask` 只在信号线程 lambda 里调，其它线程没继承 mask → `SIGTERM` 直接被别的线程收到、绕过清理成僵尸；且清理无优雅期立即 `unmount`，已 fetch segment 的 peer 后续 get/put 报错。修：把信号屏蔽挪到 `ResourceTracker` 构造函数（主线程、早于所有应用线程），后续线程都继承 mask；加 `MC_STORE_TERM_GRACE_SECONDS` 优雅期（默认 0，保持原行为）。

5. 高频通用追问（跨 PR，务必背）

- 「你怎么在没有真实多卡/RDMA 环境下验证？」→ 分两层：能编译的做单元/回归测试（storage_backend_test、standalone entropy_test）；硬件相关（RDMA 端口 flap、VMA 耗尽）靠代码路径推理 + git diff review + 已有测试覆盖，并在 PR 里诚实标注哪些是编译验证、哪些是逻辑验证。不谎报跑过硬件。
- 「判断一个 CI 红是 flaky 还是真 bug？」→ 永远先 `gh run view <id> --job <jobid> --log-failed` 读日志；主干绿不代表我的 PR 没引入真 bug（#3604 就是真编译错）；用 stash/revert 对照确认预存在失败（#3614 那个 restart 测试）。
- 「minimal fix 的取舍原则？」→ 优先不改协议、自包含、可回滚的最小修（#3652 熵 vs owner_id、#2632 future vs 线程池）；把"完整方案"作为独立设计点留给需要跨组件协同的场景。
- 「这些 bug 的共性？」→ 大多是资源生命周期 / 关键路径阻塞 / 确定性 vs 概率损坏三类：offload 与心跳共线程（阻塞关键路径）、QP/MR/endpoint 不随生命周期回收（资源泄漏）、种子撞车（确定性损坏）。面 Mooncake 组时这套归纳能体现系统 sense。

6. 待补（如果时间够）

- 简历目前没有 Mooncake 这一条——面阿里这个组，这是最大对口筹码却没上简历。建议补一条（贴 §2 全景，选可量化 / 可追溯的：如"向 Mooncake 上游提交 8 项 bugfix PR，覆盖 Store SSD offload 正确性与 Transfer Engine RDMA 资源生命周期"）。等你决定，我不擅自改简历。
- Conductor 调度器 / prediction-based early rejection 的论文细节（我目前只到架构层，没深入调度算法）——若面试偏调度可再补。